TI205 - Introduction à la complexité

CM2 - Principes de récursion et diviser pour reigner

rado.rakotonarivo@efrei.fr

Réduction

- **La réduction** est une technique utilisée pour résoudre un problème en le transformant en un autre problème pour lequel nous avons déjà un algorithme.
- Réduire le problème X au problème Y signifie écrire un algorithme qui résout X en utilisant un algorithme qui résout Y comme une boîte noire ou une fonction.

Réduction : exemple

L'algorithme `selection_sort` réduit le problème de tri d'un tableau en utilisant deux problèmes plus simples:

- Trouver l'indice du plus petit élément d'un tableau
- Échanger deux éléments d'un tableau

```
Function : selection_sort (T: array of integer, n: integer)
Local variables:
Begin
    for i ← 1 to n - 1 do
        min_index ← min(T, i, n) // Trouver l'indice du le plus petit élément
        if min_index ≠ i then
            swap(T[i], T[min_index]) // Echanger l'élément courant avec le plus petit élément
        end if
    end for
End
```

Réduction : exemple

- Il n'est pas important de savoir comment fonctionnent les fonctions `min` et `swap`.
- La seule chose qui importe est qu'elles résolvent correctement leurs *parties*.
- Mais bien sûr: **le temps d'exécution de l'algorithme dépend du temps d'exécution des ces fonctions**.
- La complexité en temps de l'algorithme `selection_sort` reste donc $n \times (\mathcal{O}(n) + \mathcal{O}(1)) = \mathcal{O}(n^2)$.

Récursivité

- **La récursivité** est un cas particulier de réduction où le problème est réduit à une version plus petite ou *instance plus petite* de lui-même.
- Résoudre un problème de manière récursive peut être décrit comme suit:
  - **Cas de base**: Si l'instance donnée peut être résolue directement, alors la résoudre directement.
  - **Cas récursif**: Sinon, réduire le problème à une ou plusieurs instances plus simples de lui-même et le résoudre récursivement.

> **Remarque**: Une instance plus petite d'un algorithme est une instance du même algorithme avec une taille d'entrée plus petite.

Exemple

La factorielle d'un entier non négatif n est définie comme suit:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

- Le cas de base est $0! = 1$.
- Le cas récursif $n! = n \times (n-1)!$.

```
Function : factorial(n: integer) → integer
Local variables:
Begin
    if n = 0 then
        return 1
    else
        return n * factorial(n - 1)
    end if
End
```

L'algorithme récursif correspondant est immédiat.

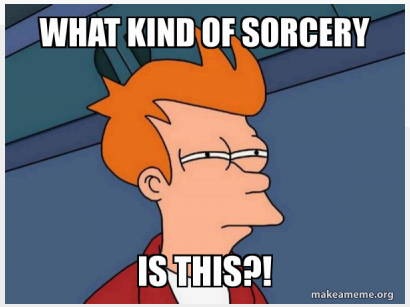
Exemple

La factorielle d'un entier non négatif n est définie comme suit:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

- Le cas de base est $0! = 1$.
- Le cas récursif
  $n! = n \times (n-1)!$.

```
Function : factorial(n: integer) → integer
Local variables:
Begin
    if n = 0 then
        return 1
    else
        return n * factorial(n - 1)
    end if
End
```



L'algorithme récursif correspondant est immédiat.

Exemple

La factorielle d'un entier non négatif n est définie comme suit:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

- Le cas de base est $0! = 1$.
- Le cas récursif
  $n! = n \times (n-1)!$.

```
Function : factorial(n: integer) → integer
Local variables:
Begin
    if n = 0 then
        return 1
    else
        return n * factorial(n - 1)
    end if
End
```

L'algorithme récursif correspondant est immédiat.

Exemple

- Si on peut se convaincre que l'algorithme `selection_sort` est correct en supposant que les fonctions `min` et `swap` font leur travail correctement, même si on ne sait pas comment elles fonctionnent.
- Alors, on peut se convaincre que l'algorithme `factorial(n)` est correct en supposant que `factorial(n-1)` fait son travail correctement, même si on ne sait pas comment il fonctionne.

Comment résoudre un problème en utilisant la récursivité ?

Supposons que nous ne disposons pas de la représentation récursive d'un problème. Comment pouvons-nous le résoudre de manière récursive ?

Problème 1

Écrire une fonction récursive `sum_rec` qui, étant donné un entier non négatif n, renvoie la somme de tous les entiers de 1 à n.

Comment résoudre un problème en utilisant la récursivité ?

Étapes à suivre

1. Trouver l'entrée la simple / la plus petite taille d'entrée possible
2. Visualiser des exemples sur plusieurs tailles d'entrée
3. Trouver une relation entre une grande instance et une instance plus petite
4. Généraliser le schéma récursif
5. Écrire l'algorithme en combinant le cas de base et le cas récursif

Comment résoudre un problème en utilisant la récursivité ?

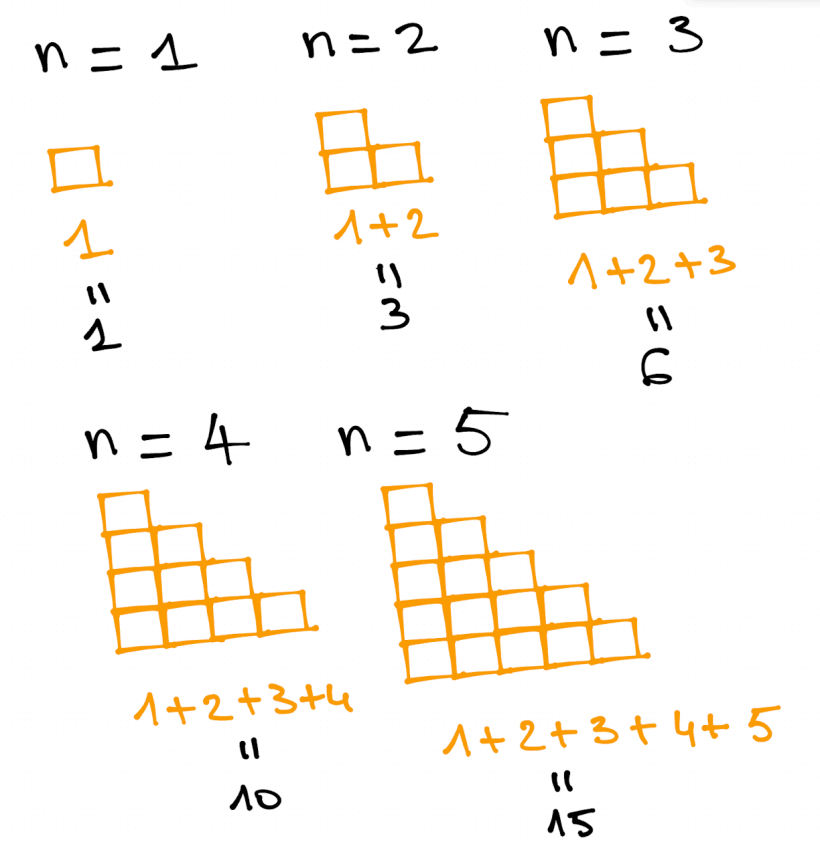
Trouver l'entrée la simple / la plus petite taille d'entrée possible

- Cette étape consiste à trouver le cas de base, c'est-à-dire la plus petite instance du problème qui peut être résolue directement.
- Pour notre cas, l'entrée la plus simple est $n = 0$. Ainsi, le cas de base est $\text{sum_rec}(0) = 0$.

Comment résoudre un problème en utilisant la récursivité ?

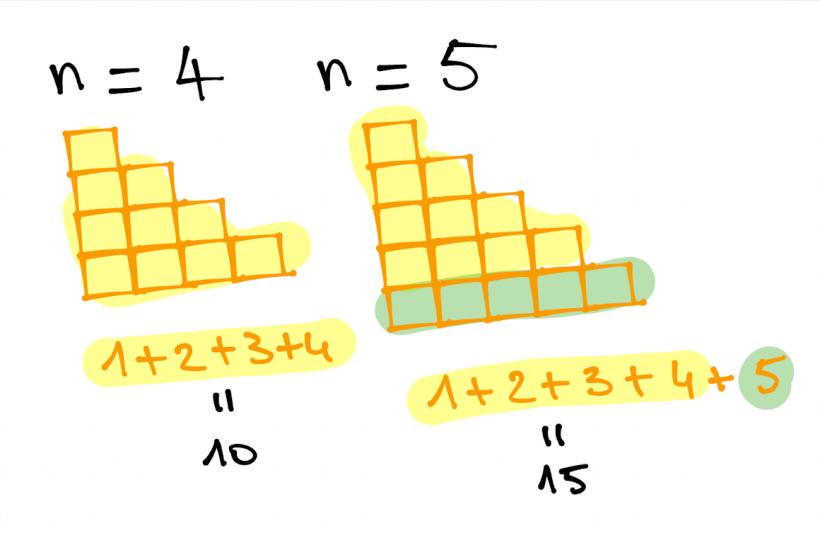
Visualiser des exemples

- Visualiser comment les entrées et les sorties changent pour différentes tailles de l'instance du problème.



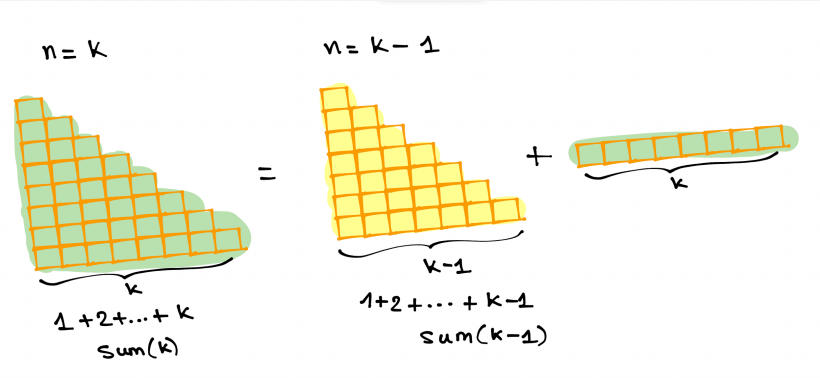
Trouver une relation entre une grande instance et une instance plus petite

- Par exemple, si vous avez la réponse pour $n = 4$, comment pouvez-vous l'utiliser pour trouver la réponse pour $n = 5$?
- Ici, nous pouvons facilement voir que $\text{sum_rec}(5) = \text{sum_rec}(4) + 5$.



Généraliser le schéma récursif

- Trouver le schéma récursif qui relie une grande instance à une instance plus petite pour n'importe quelle taille de l'entrée.
- Ici, nous pouvons voir que pour une entrée de taille k, $\text{sum_rec}(k) = \text{sum_rec}(k-1) + k$.



Comment résoudre un problème en utilisant la récursivité ?

Écrire l'algorithme

En combinant le cas de base et le cas récursif, nous obtenons la description récursive suivante de la fonction `sum_rec` .

$$\text{sum_rec}(n) = \begin{cases} 0 & \text{si } n = 0 \\ \text{sum_rec}(n-1) + n & \text{si } n > 0 \end{cases}$$

Comment résoudre un problème en utilisant la récursivité ?

Écrire l'algorithme

De cette description découle l'algorithme ainsi que le code en C suivants:

```
Function : sum_rec(n: integer) → integer
Local variables:
Begin
    if n = 0 then
        return 0
    else
        return sum_rec(n - 1) + n #  appel récursif
    end if
End
```

```c
int sum_rec(int n) {
    if (n == 0) { // cas de base
        return 0;
    } else {
        return sum_rec(n - 1) + n; // cas récursif
    }
}
```

Comment ce code fonctionne ?

- Appel de `sum_rec(3)`.
  - $3 \neq 0 \rightarrow$ retourne `sum_rec(2) + 3`.
  - Appel de `sum_rec(2)`.
    - $2 \neq 0 \rightarrow$ retourne `sum_rec(1) + 2`.
    - Appel de `sum_rec(1)`.
      - $1 \neq 0 \rightarrow$ retourne `sum_rec(0) + 1`.
      - Appel de `sum_rec(0)` **Cas de base atteint!** retourne `0`.
    - `sum_rec(1)` retourne `0 + 1 = 1`.
  - `sum_rec(2)` retourne `1 + 2 = 3`.
- `sum_rec(3)` retourne `3 + 3 = 6`.

Comment résoudre un problème en utilisant la récursivité ?

Exemple

On considère cette version itérative de la recherche binaire.

```
Function: binary_search(T: array of integer, val: integer, l: integer, r: integer) → integer
Local variables:
Begin
  while l ⩽ r do
    mid ← (r - l) / 2 + l
    if T[mid] = val then
      return mid
    else if T[mid] < val then
      l ← mid + 1
    else
      r ← mid - 1
    end if
  end while
  return -1
end
```

Comment résoudre un problème en utilisant la récursivité ?

Exemple

- Cette fonction recherche la valeur `val` dans le tableau trié `T` entre les indices `l` et `r` et retourne l'indice de la valeur si elle est trouvée, sinon elle retourne -1.
- Pour un tableau T de taille n, on fera l'appel $\mathrm{binary_search}(\mathrm{T}, \mathrm{val}, 1, \mathrm{n})$.

Problème 2

Écrire un algorithme récursif `binary_search_rec` qui résout le même problème.

Comment résoudre un problème en utilisant la récursivité ?

Identifier le/les cas de base

- Le cas le plus simple ici est lorsque le tableau est vide, c'est-à-dire $n = 0$. **Cela se traduit par $l > r$ dans notre cas**.
- Notons également que lorsque la valeur est trouvée, le problème est résolu (il n'y a pas besoin d'autres appels récursifs).

Nous avons donc deux cas de base:

- quand $l > r$, on retourne -1.
- quand $T[\text{mid}] = \text{val}$, on retourne mid.

Comment résoudre un problème en utilisant la récursivité ?

Visualiser des exemples

Comment résoudre un problème en utilisant la récursivité ?

Trouver une relation entre une grande instance et une instance plus petite

- On peut reformuler le problème comme ceci: "*On nous donne un tableau T, dont le premier index est l et le dernier index est r, chaque fois que nous ne trouvons pas la valeur à l'index mid, nous divisons le tableau en deux en mettant à jour son premier et son dernier index*".
- Une grande instance de notre problème s'écrit alors comme ceci:

$$\text{binary_search_rec}(T[l \ldots r], val)$$

- Et les instances plus petites sont:
  - $T[l \ldots \text{mid} - 1]$ si $T[\text{mid}] > val$
  - $T[\text{mid} + 1 \ldots r]$ si $T[\text{mid}] < val$

Comment résoudre un problème en utilisant la récursivité ?

Généraliser le schéma récursif

Le schéma récursif est le suivant:

$$\text{binary_search_rec}(T[l \dots r], val) = \begin{cases} -1 & \text{si } l > r \\ \text{mid} & \text{si } T[\text{mid}] = val \\ \text{binary_search_rec}(T[l \dots \text{mid} - 1], val) & \text{si } T[\text{mid}] > val \\ \text{binary_search_rec}(T[\text{mid} + 1 \dots r], val) & \text{si } T[\text{mid}] < val \end{cases}$$

Pour résoudre le problème initial, on fait l'appel de:

$$\text{binary_search_rec}(T, val, 1, n).$$

Comment résoudre un problème en utilisant la récursivité ?

Écrire l'algorithme

```
Function: binary_search(T: array of integer, val: integer, l: integer, r: integer) → integer
Local variables:
Begin
  if l > r then
    return -1
  end if
  mid ← (r - l) / 2 + l
  if T[mid] = val then
    return mid
  else if T[mid] < val then
    return binary_search_rec(T, val, mid + 1, r)
  else
    return binary_search_rec(T, val, l, mid - 1)
  end if
End
```

Comment résoudre un problème en utilisant la récursivité ?

Écrire l'algorithme

Le code en C correspondant est le suivant:

```
int binary_search_rec(int T[], int val, int l, int r) {
    if (l > r) return -1; // cas de base
    else {
        int mid = (r - l) / 2 + l;
        if (T[mid] == val) // l'autre cas de base
            return mid;
        else if (T[mid] < val)
            return binary_search_rec(T, val, mid + 1, r); // cas récursif
        else
            return binary_search_rec(T, val, l, mid - 1); // cas récursif
    }
}
```

Qu'avons-nous appris jusqu'à présent ?

- La récursivité est un cas particulier de réduction où le problème est réduit à une version plus petite de lui-même.
- Un algorithme récursif fait appel à lui-même avec une taille d'entrée plus petite.
- On peut se convaincre de la validité d'un algorithme récursif en supposant que les appels récursifs font leur travail correctement.
- Un algorithme récursif doit obligatoirement avoir un cas de base qui se résout directement, sans lequel il bouclera indéfiniment.
- Un algorithme itératif peut être transformé en un algorithme récursif.

Comment analyser la complexité des algorithmes récursifs ?

- La complexité en temps d'un algorithme récursif est déterminée par, soit:
  - **Le nombre d'appels récursifs** et **la complexité en temps de chaque appel**.
  - **Le nombre d'opérations élémentaires** effectuées par l'algorithme.
- Afin de calculer le nombre d'appels récursifs, ou le nombre d'opérations élémentaires, on peut établir une **relation de récurrence** qui exprime l'un de ces deux éléments en fonction de leur valeur pour une taille d'entrée plus petite.

> **La résolution de cette relation de récurrence permet alors d'obtenir une forme close (uniquement en fonction d'une taille d'instance) permet alors d'aboutir à complexité en temps de l'algorithme**

Comment analyser la complexité des algorithmes récursifs ?

Qu'est-ce qui différencie l'analyse de celle d'un algorithme itératif ?

Pour un algorithme itératif :

- Considérer le cas qui produit le plus grand nombre d'opérations élémentaires (*pire cas*).
- Identifier l'op. élém. qui contribue le plus au temps d'exécution.
- Compter le nombre d'opérations élémentaires de ce type pour une taille d'entrée donnée.
- Exrpimer ce nombre en notation $\mathcal{O}$.

Exemple : Complexité de `sum_rec`

- Ici il n'y a pas de pire cas, car l'algorithme effectue toujours le même nombre d'opérations pour une taille d'entrée donnée.
- L'opération qui contribue le plus à la complexité est l'addition.
- Soit $S(n)$ le nombre d'additions effectuées par l'algorithme `sum_rec` pour une entrée de taille n. On a:

$$S(n) = \begin{cases} 0 \text{ si } n = 0 \\ S(n-1) + 1 \text{ sinon} \end{cases}$$

> C'est la relation de récurrence que vérifie le nombre d'additions effectuées par l'algorithme `sum_rec` pour une entrée de taille n.

Exemple : Complexité de `sum_rec`

En déroulant la récurrence, on peut voir que:

$$
\begin{aligned}
S(n) &= S(n-1) + 1 \\
&= (S(n-2) + 1) + 1 = S(n-2) + 2 \\
&= (S(n-3) + 1) + 2 = S(n-3) + 3 \\
&= \dots \\
&= (S(n-n) + 1) + n - 1 = S(0) + n \\
S(n) &= n
\end{aligned}
$$

Donc la complexité en temps de l'algorithme `sum_rec` est $\mathcal{O}(n)$.

Exemple : Complexité de `sum_rec`

En considérant le nombre d'appels récursifs, on aurait pu écrire la relation de récurrence suivante:

$$C(n) = \begin{cases} 1 \text{ si } n = 0 \\ C(n-1) + 1 \text{ sinon} \end{cases}$$

- $C(0)$ vaut 1, car il y a un appel à la fonction `sum_rec` pour $n = 0$.
- $C(n)$ vaut $C(n-1) + 1$ car on comptabilise le nombre d'appels récursifs effectués par la fonction `sum_rec(n - 1)` et l'appel initial à `sum_rec(n)`.

> Déroulez cette récurrence pour prouver qu'on obtient la même compléxité en temps que précédemment.

Exemple : Complexité de `binary_search_rec`

- Le pire cas est lorsque la valeur n'est pas dans le tableau.
- Considérons la comparaison $T[\text{mid}] = \text{val}$ comme l'opération qui contribue le plus au temps d'exécution.
- Soit $B(n)$ le nombre de comparaisons de ce type effectué par l'algorithme `binary_search_rec` sur un tableau de taille n. **Nous allons supposer que n est une puissance de 2, c'est-à-dire $n = 2^k$.**
- On a:

$$B(n) = \begin{cases} 0 \text{ si } n = 0 \\ 1 \text{ si } n = 1 \\ B(n/2) + 1 \text{ sinon} \end{cases}$$

Exemple : Complexité de `binary_search_rec`

En déroulant la récurrence, on peut voir que:

$$\begin{aligned} B(n) &= B(n/2) + 1 \\ &= (B(n/4) + 1) + 1 = B(n/2^2) + 2 \\ &= (B(n/8) + 1) + 2 = B(n/2^3) + 3 \\ &= \dots \\ B(n) &= B(n/2^k) + k \end{aligned}$$

Comme $n = 2^k$, nous avons $k = \log_2(n)$. Donc:

$$B(n) = B(1) + \log_2(n) = 1 + \log_2(n)$$

Ainsi, la complexité en temps de l'algorithme `binary_search_rec` est $\mathcal{O}(\log_2 n)$.

Diviser pour régner

L'approche **Diviser pour régner** est un paradigme de conception d'algorithmes qui résout un problème en suivant le principe suivant:

- **Diviser**: Diviser le problème en plusieurs sous-problèmes plus petits et indépendants.
- **Régner**: Résoudre les sous-problèmes récursivement.
- **Combiner**: Combiner les solutions des sous-problèmes pour résoudre le problème initial.

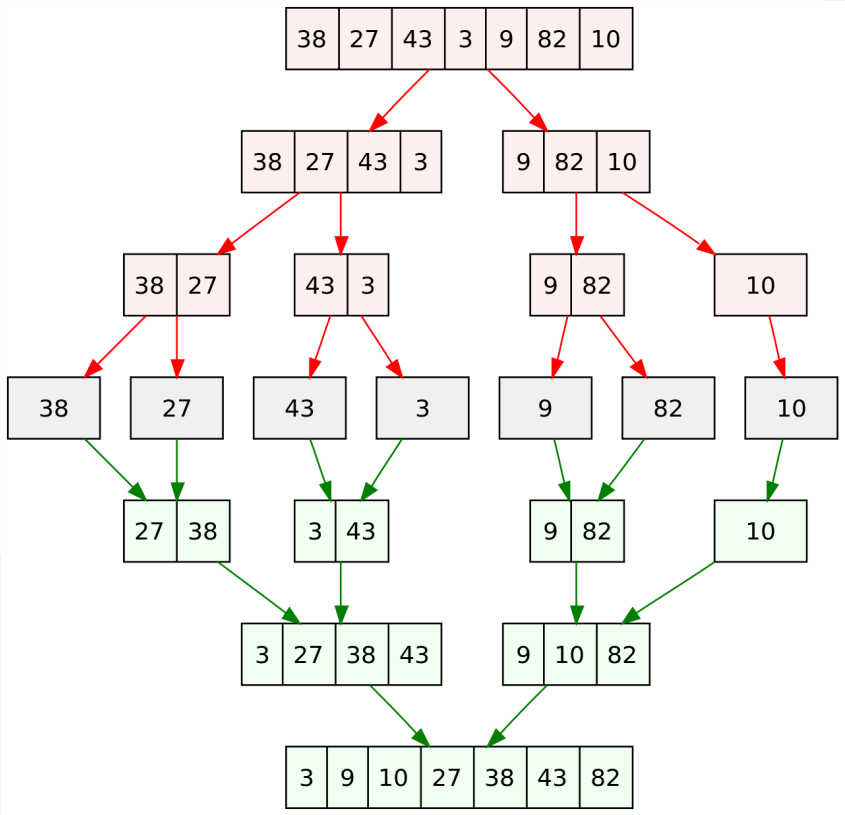
> **Remarque**: Si le problème peut être résolu directement, pour une certaine taille de l'entrée, alors il faut le résoudre directement.

Exemple : Le tri fusion (algorithme)

- Le tri fusion est un algorithme de tri qui utilise le paradigme diviser pour régner.
- L'algorithme divise le tableau en deux moitiés, trie les deux moitiés récursivement, puis fusionne les deux moitiés triées.

```
Function : merge_sort(T : array of integer, l: integer, r: integer)
Begin
  if l < r then
    mid ← (r - l) / 2 + l          # divide
    merge_sort(T, l, mid)          # conquer (left recursion)
    merge_sort(T, mid + 1, r)      # conquer (right recursion)
    merge(T, l, mid, r)            # combine
  end if
End
```

C'est la fonction `merge` qui combine les deux moitiés triées en un seul tableau trié. Elle s'exécute en $\mathcal{O}(n)$ (Elle sera vue en TP).


38 27 43 3 9 82 10
38 27 43 3
9 82 10
38 27
43 3
9 82
10
38
27
43
3
9
82
10
27 38
3 43
9 82
10
3 27 38 43
9 10 82
3 9 10 27 38 43 82

Exemple : Le tri fusion (complexité)

> **Remarque**: Nous pouvons avoir des problèmes qui nécessitent plus de réflexion pour déterminer la complexité en temps.

- Ici le pire cas n'est pas évident : **ce n'est pas le cas où le tableau est trié dans l'ordre inverse**. (Pourquoi ?)
- L'op. élémentaire qui contribue le plus au temps d'exécution semble cachée. (*HINT*: Elle est dans la fonction `merge`).
- Mais comme `merge_sort` est un algorithme récursif, nous pouvons utiliser une relation de récurrence pour déterminer sa complexité en temps!

Exemple : Le tri fusion (complexité)

Remarques: Il nous est donné que `merge` s'exécute en $\mathcal{O}(n)$.

- Le cas de base est lorsque $l \geq r$, c'est-à-dire que le tableau a une taille de 1 (dans ce cas le tableau est déjà trié) ou 0. Dans les deux cas, il n'y a rien à faire.
- Soit $M(n)$ le temps d'exécution de l'algorithme `merge_sort` sur un tableau de taille n (que l'on supposera puissance de 2). Nous avons:

$$M(n) = \begin{cases} c \text{ si } n \leq 1 \\ 2M(n/2) + cn \text{ si n} > 1 \end{cases}, \text{ où } c \text{ est une constante positive.}$$

Exemple : Le tri fusion (complexité)

En déroulant la récurrence, on peut voir que:

$$\begin{aligned} M(n) &= 2M(n/2) + cn \\ &= 2(2M(n/4) + c(n/2)) + cn = 4M(n/4) + 2cn \\ &= 4(2M(n/8) + c(n/4)) + 2cn = 8M(n/8) + 3cn \\ &= \dots \\ M(n) &= 2^k M(n/2^k) + kcn \end{aligned}$$

Comme $n = 2^k$, nous avons $k = \log_2(n)$. Donc:

$$M(n) = nM(1) + cn\log_2 n = cn + cn\log_2 n = \mathcal{O}(n\log_2 n)$$

Remarques supplémentaires

1. Afin de simplifier les calcus, on a supposé que $n = 2^k$. Ce n'est pas toujours le cas.
   - Pour le tri fusion, la complexité en temps est $\mathcal{O}(n \log_2 n)$, même si la taille de l'entrée n'est pas une puissance de 2.
   - En vrai, la relation de récurrence est $M(n) = M(\lfloor n/2 \rfloor) + M(\lceil n/2 \rceil) + cn$.
   - Mais comme on cherche une borne supérieure, nous pouvons surestimer $M(n)$ et obtenir le même résultat. (Des calculs supplémentaires permettent de prouver que notre simplification est correcte).
2. Il existe d'autres méthodes pour résoudre la relation de récurrence:
   - La **méthode de substitution**. (C'est ce que nous avons utilisé dans les exemples précédents)
   - La **méthode des arbres de récurrence**.
   - Le **master theorem**.

Conclusion

- La récursivité est une technique puissante pour résoudre des problèmes en les réduisant à des instances plus petites du même problème.
- L'approche diviser pour régner est un paradigme de conception d'algorithmes qui utilise la récursivité.
- La complexité en temps d'un algorithme récursif est souvent exprimée à l'aide d'une relation de récurrence. Résoudre cette relation de récurrence permet de déterminer la complexité de l'algorithme.
- La récursivité n'est pas limitée aux algorithmes. Elle s'applique également aux structures de données (listes chaînées, arbres, graphes, etc.).

Conclusion

Pourtant

- Une fois transposé dans un langage de programmation, un algorithme récursif peut être moins efficace que son équivalent itératif.
- Les appels récursifs peuvent être coûteux en termes de d'espace mémoire et peuvent entraîner un *débordement de pile* si la profondeur de la récursion est trop grande.
- Il faut donc être prudent lors de l'utilisation de la récursivité.